CS452 Midterm Status Report

Christos Michail, Andreas Nearchou · 6 November 2026

Project: Resource Isolation and Cold Start Latency in Serverless Runtimes (Docker vs Wasmtime).

Progress Made

Design, protocol, implementation and a first pilot are done (preliminary).

- **Design:** a CPU-bound prime counter (sieve) built from one Rust source as a native binary (Docker) and as `wasm32-wasip1` (Wasmtime). Stdin/stdout protocol: `READY`, `RUN N`, `RESULT count`, `QUIT`. Metrics: cold startup, warm latency, peak memory.
- **Implementation:** unit-tested Rust app (168 for N=1000, 1229 for N=10000). The Wasm build (Wasmtime 50.0.1) and Andreas's Docker container return identical output.
- **Pilot** (same WSL2 host, N=10000, 5 cold and 10 warm trials): Wasmtime median cold startup about 18 ms (compile cache on) and 55 ms (off), warm 0.24-0.28 ms. Docker median cold startup about 876 ms, warm about 1.23 ms.

Challenges

The main risk is a fair comparison between the runtimes.

- **Fair comparison:** Docker includes daemon and container creation, Wasmtime may compile per run. We will record the cache setting and host, and agree what cold means.
- **Peak memory:** runtimes expose memory differently (cgroup vs process), so we need one common metric.
- **Variance:** the first cold trial was slower, so we will repeat trials.

Next Steps

Remaining work:

1. Complete the Docker and Wasmtime runners behind one interface, add the timing controller and `memory.py`, and agree host, trial counts and cache policy.
2. Run the final experiments (cold and warm), compute mean, median, std and P95 with pandas, and plot with matplotlib.
3. Finalise conclusions and limitations and deliver the presentation.